Vectorizing Sparse Coiteration for Two-finger Loop Structure (Extended Abstract)

Anonymous Author(s)

1 Introduction and Motivation

While dense tensor optimizations are well-established [2, 5, 6, 8, 10, 14, 14], the shift toward sparse formats complicates iteration and automated code generation [3, 4, 7, 9, 11–13]. To address this, tensor compilers increasingly rely on the Compressed Sparse Fiber (CSF) format for its generalizability [11], which models tensor storage hierarchically. CSF allows each axis to be independently annotated as dense (relying on implicit indexing) or compressed (using explicit position and coordinate arrays to track non-zeros).

To navigate the complexities of generating code for diverse sparse data layouts, advanced tensor compilers have been developed. Most notably, building upon the pioneering success of TACO [7], the MLIR framework integrated a dedicated sparse tensor dialect [1] that adapts TACO's principles for a modern, multi-level compilation ecosystem. Consequently, active research is heavily focused on designing and implementing novel optimization strategies across the various abstraction layers within this infrastructure.

This infrastructure is fundamentally driven by a progressive lowering pipeline. The `SparseTensor` dialect seamlessly integrates with MLIR's broader ecosystem, orchestrating transformations down through the Linear-Algebra (`Linalg`), Structured-Control-Flow (`SCF`), and `Vector` dialects (Figure 1). At the highest level, core computations are abstractly defined using `linalg.generic` operations and coupled with per-tensor sparse encoding annotations. As the compiler lowers this representation, the sparsification pipeline targets the `Vector` dialect to automatically exploit SIMD vectorization wherever the underlying data layout and hardware constraints permit.

However, MLIR currently restricts this vectorization strictly to statically predictable dense axes within the CSF format, completely bypassing profitable vectorization opportunities for compressed dimensions. This conservative approach stems from a fundamental challenge in sparse tensor compilation: execution efficiency is inherently determined by the input dynamic non-zero structure of data. Because traditional static compile-time transformations cannot predict this runtime sparsity distribution, they fundamentally fail to exploit data-dependent optimization opportunities.

A critical example of this limitation manifests in the *two-finger* `scf.while` loop structures generated during the lowering process. This co-iteration loop is a fundamental characteristic of sparse tensor computations when a shared dimension is compressed in both input tensors. To traverse this space, the compiler must simultaneously advance two

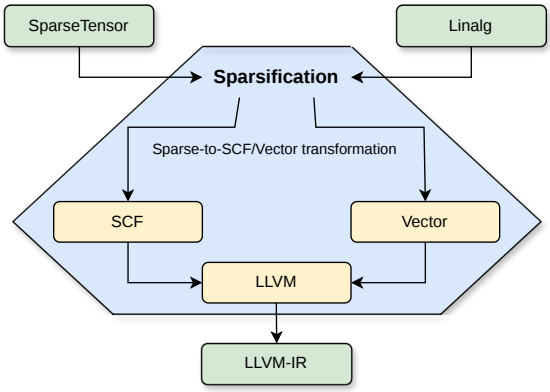


Figure 1. The standard MLIR sparse lowering pipeline integrates the `Linalg` dialect for computational logic with the `SparseTensor` dialect for storage metadata through a specialized *Sparsification* pass.

cursors through their respective coordinate arrays. Crucially, for intersection operations, the inner computation is strictly predicated on a coordinate match; it executes exclusively when both cursors point to the same index, confirming the presence of intersecting non-zero values. Because these pointers must advance dynamically based on runtime coordinate comparisons, this data-dependent control flow inherently limits standard loop optimizations, most notably vectorization [4, 7].

2 Approach

The introduction of divergent `if/else` control flow within the `scf.while` loop inherently disrupts standard SIMD auto-vectorization. A naive alternative to circumvent this divergence is to aggressively evaluate the co-iteration space as a dense kernel by logically materializing the compressed tensors with explicit zeros. While this flattens the execution into a perfectly vectorizable, branchless, affine loop spanning the full coordinate bounds, the resulting overhead of performing redundant arithmetic on structural zeros unacceptably negates both the memory and performance benefits of a sparse representation.

In this work, we demonstrate that the immense throughput advantages of vectorized execution can actually outweigh the penalty of processing a small fraction of structural zeros. To achieve this, we transform the standard co-iteration into a dual-path architecture consisting of a vectorized fast-path and a scalar slow-path (Refer to Figure 2). The fast-path

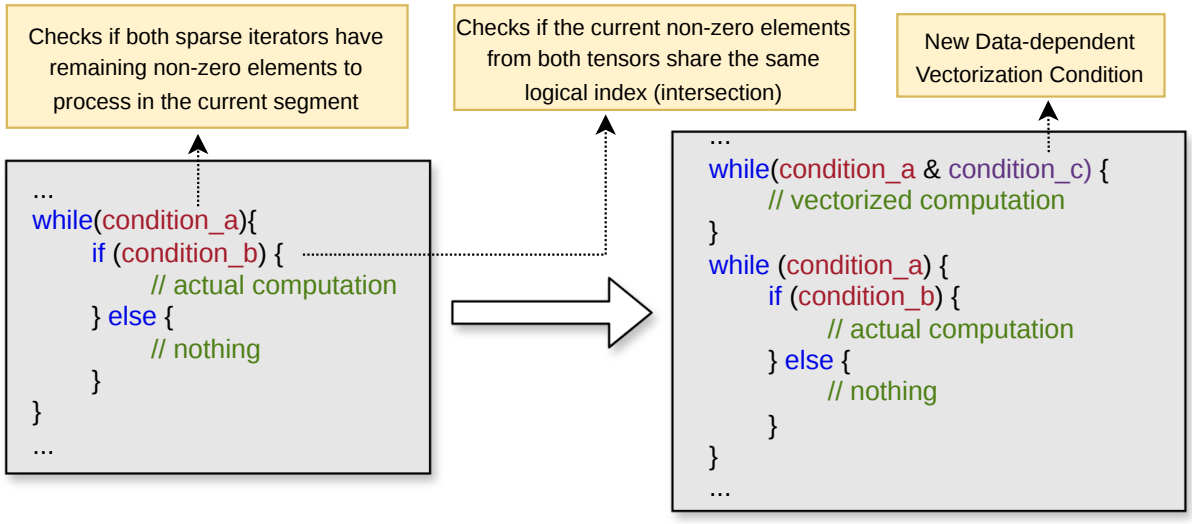


Figure 2. Transformation of the standard scalar co-iteration loop (left) into our proposed dual-path vectorized architecture (right). The fast lane executes fixed-width SIMD operations guarded by a new data-dependent bounds check (`condition_c`), while the original loop is retained as a scalar epilogue to process remaining tail elements.

Table 1. Basic Sparse Kernels with Randomly Generated Data with only 5% Non-zero values

Kernel	Equation	Baseline (s)	Optimized (s)	Speedup×
SpGeMM	$C_{ik} = \sum_i A_{ij} \cdot B_{jk}$	16.01	8.02	2×
SpMV	$y_i = \sum_j A_{ij} \cdot x_j$	179.59	104.97	1.71×
MTTKRP	$A_{ij} = \sum_{l,k} B_{ikl} \cdot C_{kj} \cdot D_{lj}$	845.79	418.73	2.02×

Table 2. Performance and Speedup of Higher-order Tensor Contraction Kernels on 1% Non-zero Values

Kernel Name & Equation	Baseline (s)	Optimized (s)	Speedup
1. DMRG Effective Hamiltonian $E = L(a,d,x) \cdot \text{Bra}(a,b,c) \cdot W(d,e,c,f) \cdot \text{Ket}(x,y,f) \cdot R(b,e,y)$	446.66	323.03	1.38×
2. DMRG Two-Site Update $\Theta(a,c,f,y) = L(a,d,x) \cdot W_1(d,e,c,s) \cdot W_2(e,g,f,t) \cdot \text{Ket}(x,s,t,y) \cdot R(y,g,b)$	2948.46	1789.23	1.65×

iterates at the hardware's physical vector width, utilizing predicated execution and vector masks to safely perform speculative SIMD operations and filter out mismatched zero-computations. A dynamically evaluated, data-dependent runtime condition strictly routes execution to this fast path whenever sufficient non-zeros remain, relying on the original scalar `scf.while` loop solely as a slow-path epilogue to process remaining tail elements.

3 Evaluation

Our initial hypothesis posited that fundamental sparse operations such as SpGEMM and SpMV would represent worst-case scenarios for vectorization due to their low arithmetic intensity. We anticipated that the high latency of masked gather operations would outweigh the performance gains of SIMD execution for simple multiply-accumulate operations. Consequently, we designed our evaluation suite to prioritize high-order tensor contractions from quantum computational physics, such as the Density Matrix Renormalization Group (DMRG) effective Hamiltonian contraction:

$$E = L(a,d,x) \cdot \text{Bra}(a,b,c) \cdot W(d,e,c,f) \cdot \text{Ket}(x,y,f) \cdot R(b,e,f)$$

However, empirical evaluation contradicted these theoretical constraints by demonstrating marked performance improvements across both kernel categories. This reveals that our approach's high instruction throughput effectively absorbs gather latencies—even in memory-bound, low-intensity environments—proving its profitability across diverse sparse workloads.

References

[1] Aart Bik, Penporn Koanantakool, Tatiana Shpeisman, Nicolas Vasilache, Bixia Zheng, and Fredrik Kjolstad. 2022. Compiler Support for Sparse Tensor Computations in MLIR. 19, 4, Article 50 (Sept. 2022), 25 pages. doi:10.1145/3544559

[2] Tianqi Chen, Lianmin Zheng, Eddie Q. Yan, Ziheng Jiang, T. Moreau, L. Ceze, Carlos Guestrin, and A. Krishnamurthy. 2018. Learning to Optimize Tensor Programs. (2018), 3393–3404.

[3] Yuedan Chen, Kenli Li, Wangdong Yang, Guoqing Xiao, Xianghui Xie, and Tao Li. 2019. Performance-Aware Model for Sparse Matrix-Matrix Multiplication on the Sunway TaihuLight Supercomputer. *IEEE Transactions on Parallel and Distributed Systems* 30 (2019), 923–938. doi:10.1109/tpds.2018.2871189

[4] Stephen Chou, Fredrik Kjolstad, and Saman P. Amarasinghe. 2018. Format abstraction for sparse tensor algebra compilers. *Proceedings of the ACM on Programming Languages* 2 (2018), 1 – 30. doi:10.1145/3276493

[5] Mahdi Ghorbani, Emilien Bauer, Tobias Grosser, and Amir Shaikhha. 2025. Compressed and Parallelized Structured Tensor Algebra. *Proc. ACM Program. Lang.* 9, OOPSLA1, Article 141 (April 2025), 29 pages. doi:10.1145/3720506

[6] Mahdi Ghorbani, Mathieu Huot, Shideh Hashemian, and Amir Shaikhha. 2023. Compiling Structured Tensor Algebra. 7, OOPSLA2, Article 229 (Oct. 2023), 30 pages. doi:10.1145/3622804

[7] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. 2017. The tensor algebra compiler. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 77 (Oct. 2017), 29 pages. doi:10.1145/3133901

[8] Rui Li, Aravind Sukumaran-Rajam, R. Veras, Tze Meng Low, F. Rastello, A. Rountev, and P. Sadayappan. 2019. Analytical Cache Modeling and Tilesize Optimization for Tensor Contractions. *SC19: International Conference for High Performance Computing, Networking, Storage and Analysis* (2019), 1–13. doi:10.1145/3295500.3356218

[9] Eric Qin, Geonhwa Jeong, William Won, Sheng-Chun Kao, Hyoukjun Kwon, S. Srinivasan, Dipankar Das, G. Moon, S. Rajamanickam, and T. Krishna. 2021. Extending Sparse Tensor Accelerators to Support Multiple Compression Formats. *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)* (2021), 1014–1024. doi:10.1109/ipdps49936.2021.00110

[10] Ryan Senanayake, Changwan Hong, Ziheng Wang, Amalee Wilson, Stephen Chou, Shoaib Kamil, Saman P. Amarasinghe, and Fredrik Kjolstad. 2020. A sparse iteration space transformation framework for sparse tensor algebra. *Proceedings of the ACM on Programming Languages* 4 (2020), 1 – 30. doi:10.1145/3428226

[11] Shaden Smith and George Karypis. 2015. Tensor-matrix products with a compressed sparse tensor. In *Proceedings of the 5th Workshop on Irregular Applications: Architectures and Algorithms* (Austin, Texas) *(IA³ '15)*. Association for Computing Machinery, New York, NY, USA, Article 5, 7 pages. doi:10.1145/2833179.2833183

[12] Shaden Smith and G. Karypis. 2015. Tensor-matrix products with a compressed sparse tensor. *Proceedings of the 5th Workshop on Irregular Applications: Architectures and Algorithms* (2015). doi:10.1145/2833179.2833183

[13] Jeremiah Willcock and A. Lumsdaine. 2006. Accelerating sparse matrix computations via data compression. (2006), 307–316. doi:10.1145/1183401.1183444

[14] Rohan Yadav, A. Aiken, and Fredrik Kjolstad. 2022. DISTAL: the distributed tensor algebra compiler. *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (2022). doi:10.1145/3519939.3523437